



ELEC3609/9609 - Week 4 Lab

04 Django ORM, Django Model and Django Admin

In this lab, you will learn how to define data models using Django's ORM, manage database schema through migrations, and interact with model data using the Django admin interface. The lab also introduces model relationships and provides guidance on maintaining consistency between your Python models and the underlying database.

Individual Task: [Individual Task 4 - Django Relationship Constraint and Design](#)

Lab Resource: [TIL-Lab04 \(Today I Learned project\) source code.](#)

Learning Objectives

After completing this lab, students should be able to:

- Understand the purpose and benefits of using an ORM.
- Define and manage Django models and their relationships.
- Apply and inspect schema migrations.
- Interpret Django's model-to-SQL translation.
- Use the Django admin interface to manage data.

Chapter 1. Object-Relational Mapping

ORM (Object-Relational Mapping) is a programming technique used to convert data between incompatible systems (in particular, between object-oriented programming languages and relational databases). ORMs allow developers to interact with a database using their preferred programming language, rather than writing raw SQL queries.

Key Features of ORM:

- **Mapping Objects to Database Tables:** In an ORM, each class in the programming language (e.g., Python, Java, C#) corresponds to a table in the database. Each instance of a class represents a row in that table, and each class attribute corresponds to a column in the table.
- **Abstracts SQL:** ORMs abstract away the SQL syntax and provide methods to perform common database operations (CREATE, READ, UPDATE, DELETE) using object-oriented code.
- **Automatic Data Conversion:** The ORM handles the conversion between the database types (like VARCHAR, INT) and the programming language types (like strings, integers).
- **Database-Agnostic Code:** Code written with an ORM is often database-agnostic, meaning it can work with different types of databases (e.g., MySQL, PostgreSQL, SQLite) with minimal changes. This increases flexibility and portability.
- **Reduces Boilerplate Code:** ORMs help reduce the amount of boilerplate code that developers need to write for database interactions, speeding up development and reducing the likelihood of SQL-related errors.

! INFO

Lots of ORM libraries here: Java – [Hibernate](#), Python – [Django ORM](#).



DON'T DROP DOWN TO RAW SQL UNTIL IT'S NECESSARY

Databases play a critical role in application performance, scalability, and security. They support high availability and fault tolerance, and are essential in sensitive fields like finance. Because of their complexity, managing them often requires expert teams, so use ORMs when possible, and only drop to raw SQL when absolutely needed.

ORM Can:

- **DRY Principle:** You define your data model in one place, making the code easier to update, maintain, and reuse.
- **Encourages MVC Architecture:** Promotes cleaner, more organised code by encouraging the MVC pattern.
- **Avoids Poorly Formed SQL:** Minimises errors in SQL by allowing developers to work in their native programming language. (Most web programmers are bad at it!)
- **Simplifies Security Practices:** Makes it easy to use prepared statements and transactions, enhancing security.
- **Database Abstraction:** Allows for easy changes to the underlying database system

without rewriting queries.

- **Loose Coupling:** The data model is loosely bound to the application, allowing flexibility and reuse.
- **Supports OOP Concepts:** Facilitates object-oriented programming (OOP) features like data inheritance.

ORM Cannot:

- **Learning Curve:** ORMs can be complex and are not lightweight, requiring time to learn and set up.
- **Performance Limitations:** While sufficient for typical queries, ORM-generated queries may not match the efficiency of hand-written SQL in large-scale projects.
- **Impedance Mismatch:** The way a developer uses objects is different from how data is stored and joined in relational tables.



AVOID WRITING YOUR OWN ORM FOR AN APPLICATION UNLESS IT'S SPECIFICALLY FOR LEARNING PURPOSES.

Existing ORM libraries are well-tested, optimised, and provide extensive features that are difficult to replicate effectively from scratch.

Chapter 2. Models

Models in Django are implemented via Python classes and these models form the canonical representation of our entities. The models are used to generate the underlying database schema as well as Python-Database interfaces for accessing these database tables.

For our examples in this lab, we will be placing all our models in a single Python file

`models.py`. In your assignments, you might consider splitting the models into separate files when models file becomes too large or complex (20+).



MODEL INHERITANCE

Model inheritance in Django works almost identically to the way normal class inheritance works in Python, but the basics at the beginning of the page should still be followed.

So, each model that we want to include in our application will need to subclass the

`django.db.models.Model` class. The attributes of our Python class become database fields.

Be careful with model inheritance, read more about [Model inheritance](#) in Django.

2.1 Field Types

Each attribute of our model needs to be mapped to an appropriate Django Field class. This allows Django to know the type of each attribute and generate an accurate schema. These fields can be further customised to set default values, unique constraints and validations.

For example, to define a `Post` model with a title and content:

```
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=100)
    content = models.TextField()
```

Common Django Model Field Types

Field Type	Description	Example Usage
<code>CharField</code>	A string field with a max length.	Name, title, short text fields
<code>TextField</code>	A long text field without a max length (in database).	Article content, comments
<code>IntegerField</code>	An integer field.	Age, count, score
<code>FloatField</code>	A floating-point number field.	Rating, price
<code>BooleanField</code>	A true/false field.	Is active, is published
<code>DateField</code>	A date field (without time).	Birthday, published date

Field Type	Description	Example Usage
<code>DateTimeField</code>	A date and time field.	Created timestamp, updated timestamp
<code>EmailField</code>	A string field for email addresses (with validation).	User email
<code>URLField</code>	A string field for URLs (with validation).	Profile URL, external link
<code>ImageField</code>	A field for uploading image files.	User avatar, post image
<code>FileField</code>	A field for uploading any file.	Attachment, document
<code>ForeignKey</code>	A many-to-one relationship to another model.	Author of a post, category of a product
<code>ManyToManyField</code>	A many-to-many relationship to another model.	Tags, group memberships
<code>OneToOneField</code>	A one-to-one relationship to another model.	User profile

See [Field Types](#) for the types that are supported by Django and the range of customisations that can be applied to them.

**TIP**

It is also possible to add our own types, either directly by writing a new Field subclass or through Django plugins.

2.2 Relationship Field Types

We can represent the relationships between our models using the following field types:

- **many-to-one relationship:** `models.ForeignKey` on the many-side model.

```

# An Author can write many Book instances, but each Book has only one
# Author.

# A Student can enroll in many Course instances, and a Course can have many
# Student instances.
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()

    def __str__(self):
        return self.name

class Course(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField()
    students = models.ManyToManyField(Student, related_name='courses') #
    Many-to-many

    def __str__(self):
        return self.name

```

- **one-to-one relationship:** `models.OneToOneField` or `models.ForeignKey(unique=True)`

```

# A User can have exactly one UserProfile, and a UserProfile can belong to
# exactly one User.
from django.db import models
from django.contrib.auth.models import User

class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE) # One-to-
    one
    bio = models.TextField()
    profile_picture = models.ImageField(upload_to='profiles/')

    def __str__(self):
        return self.user.username

class UserProfile(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, unique=True) #
    One-to-one alternative
    bio = models.TextField()
    profile_picture = models.ImageField(upload_to='profiles/')

```

```
def __str__(self):  
    return self.user.username
```

- **many-to-optional-one relationship:** `models.ForeignKey(null=True, blank=True)` on the many-side model.

An Employee can belong to one Department, but not every Employee has to be assigned to a department.

```
from django.db import models
```

```
class Department(models.Model):  
    name = models.CharField(max_length=100)
```

```
def __str__(self):  
    return self.name
```

```
class Employee(models.Model):  
    name = models.CharField(max_length=100)  
    department = models.ForeignKey(Department, on_delete=models.SET_NULL,  
null=True, blank=True) # Many-to-zero
```

```
def __str__(self):  
    return self.name
```

2.3 Direct Class Reference and String Reference

When defining relationships between models in Django, you have the option to refer to a related model in two ways: **Directly by Class Name** vs. **Using a String Reference**.

Referring Directly by Class Name

If the related model is already defined in the file, you can directly refer to it by its class name. This is straightforward and often used when there is no circular dependency or when the order of model definitions can be easily managed.

```
class Tag(models.Model):  
    name = models.CharField(max_length=100)  
  
class Post(models.Model):  
    tag = models.ForeignKey(Tag, on_delete=models.CASCADE) # Direct
```

reference to the Tag class

Referring to a Model Using a String Reference

If the related model is not yet defined, you can refer to it using a string containing the model's name. This allows you to define models in any order, including scenarios where two models refer to each other (mutual or circular references).

```
class Post(models.Model):
    tag = models.ForeignKey("Tag", on_delete=models.CASCADE) # String
    reference to the Tag class

# Defined Later
class Tag(models.Model):
    name = models.CharField(max_length=100)
```



LAZY RELATIONSHIPS

It also works where a model wants to reference itself. Read more about **string reference** (also known as **lazy relationships**) in Django [here](#).

Circular or Two-Way Dependency

If two models refer to each other, reordering their definitions won't solve the issue because both models depend on each other. In such cases, you use a string reference to break the circular dependency.

```
class Author(models.Model):
    name = models.CharField(max_length=100)
    # This line will cause an error if 'Book' is not yet defined
    favorite_book = models.ForeignKey(Book, on_delete=models.CASCADE,
    null=False)
    # Solution
    favorite_book = models.ForeignKey("Book", on_delete=models.CASCADE,
    null=False) # String reference

class Book(models.Model):
    title = models.CharField(max_length=200)
    author = models.ForeignKey(Author, on_delete=models.CASCADE) #
    null=False by default
```

🔥 THE EXAMPLE IS A DEADLOCK

Avoid circular FKs where both sides are required. Making `Book.author` and `Author.favorite_book` non-nullable creates a chicken-and-egg situation: you can't create either row first.

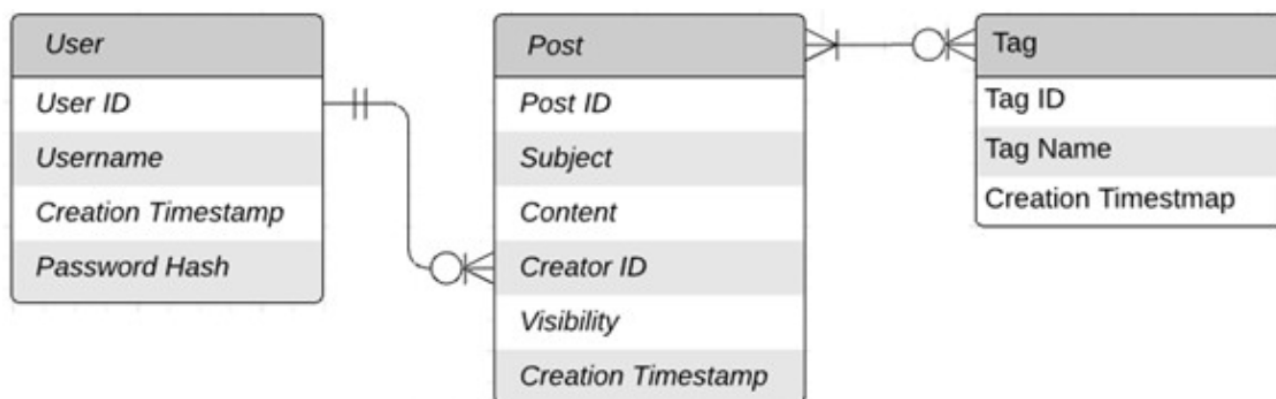
⚠️ AVOID CIRCULAR DEPENDENCY

Circular references in Django models are not inherently invalid, but they should be used with caution. While Django supports them via string references and migration strategies, they can lead to complex migration dependencies and maintenance challenges.

This complexity arises because Django models eventually map to a **relational database**, which follows a **static schema** and enforces **referential integrity**. Unlike runtime memory where objects can point to each other freely, relational databases require all references to be well-defined ahead of time. This makes circular references harder to manage and often a sign that your model design could be simplified.

2.4 TIL Model

As a reminder, here is the ERD for today's TIL example:



where a `Post` can be associated with **zero, one or many** `Tag`s. A `Tag` can be applied to **one or many** `Post`s.

⚠️ WARNING

This has not been normalised to show mapping tables (many-to-many between Post and

Tag).

Turning the TIL ERD into models will give us the following Python classes, add following in

`til_app/models.py`:

```
# til_app/models.py
from django.db import models

class Tag(models.Model):
    name = models.CharField(max_length=10, unique=True)
    creation_date = models.DateTimeField(auto_now_add=True)

class Post(models.Model):
    subject = models.CharField(max_length=160)
    content = models.CharField(max_length=800)
    # creator = ??? # Django built-in auth user
    visibility = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)

    tags = models.ManyToManyField(Tag, related_name='posts', blank=True) #
    Many-to-many relationship

    # With related_name, we can do things like
    # >>> tag_instance.posts.all() # Returns all posts associated with a
    Tag instance.
```

! INFO

For now, we'll leave the creator of a `Post` out, as we'll be using an in-built Django system for users provided by the auth application in the next lab.

💡 TIP

`related_name` is the name to use for the relation from the related object back to this one. See the [related objects documentation](#) for a full explanation and example.

⚠ WARNING

The `ManyToManyField` in Django does not provide built-in support for enforcing constraints like **"at least one."** To ensure that a `Tag` is always associated with one or more `Posts`, you need to implement custom validations and use signals. These will be

covered in a later lab.

While Django's `ManyToManyField` automatically creates an intermediary model for managing many-to-many relationships, it may lack the flexibility required for complex use cases. When more control is needed over the relationship, such as adding extra fields or enforcing custom constraints, it is often better to define an explicit mapping class instead of relying on `ManyToManyField`.

2.5 Activation

Now that we have defined our models, it is time to integrate into our project.

Django maintains a list of installed applications, defined in `settings.py` in the `INSTALLED_APPS` variable. By default, Django comes with the following applications installed:

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
]
```

Before we can generate the schema for our models, we need to add our `til` application into the list of installed applications for the project. This can be done by including the `TilConfig` class, defined in `apps.py`.

```
INSTALLED_APPS = [  
    'til_app.apps.TilAppConfig',  
    'django.contrib.admin',  
    'django.contrib.auth',  
    # ...  
]
```

Chapter 3. Schema Migrations

Django has the ability to generate and manage the database schema from the Python model files. Now that we have defined the models for our application and added the application into the list of installed applications for the project, we will be able to use the Django internal tools to generate schemas.

3.1 Make Migrations

This can be done through:

```
python3 manage.py makemigrations <application>
```

With application name, Django creates migrations only for the specified app, or you can:

```
python3 manage.py makemigrations
```

Django checks all installed apps listed in your `INSTALLED_APPS` setting and creates migrations for all apps that have pending changes.

For our example:

```
$ python3 manage.py makemigrations til_app
Migrations for 'til_app':
  til_app/migrations/0001_initial.py
    + Create model Tag
    + Create model Post
```

This will generate a migration file, but will not apply it yet. For reference, this file is located at `til_app/migrations/0001_initial.py`

3.2 View Migrations

If we want to see the corresponding commands that will be run against the database to apply

these migrations, we can use the `sqlmigrate` command:

```
python manage.py sqlmigrate til_app 0001
```

```
BEGIN;

--
-- Create model Tag
--
CREATE TABLE "til_app_tag" (
    "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    "name" VARCHAR(10) NOT NULL UNIQUE,
    "creation_date" DATETIME NOT NULL
);

--
-- Create model Post
--
CREATE TABLE "til_app_post" (
    "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    "subject" VARCHAR(160) NOT NULL,
    "content" VARCHAR(800) NOT NULL,
    "visibility" BOOL NOT NULL,
    "created_at" DATETIME NOT NULL
);

--
-- Create many-to-many table for Post and Tag relationship
--
CREATE TABLE "til_app_post_tags" (
    "id" INTEGER NOT NULL PRIMARY KEY AUTOINCREMENT,
    "post_id" BIGINT NOT NULL REFERENCES "til_app_post" ("id") DEFERRABLE
    INITIALLY DEFERRED,
    "tag_id" BIGINT NOT NULL REFERENCES "til_app_tag" ("id") DEFERRABLE
    INITIALLY DEFERRED
);

--
-- Create a unique index for the combination of post_id and tag_id
--
CREATE UNIQUE INDEX "til_app_post_tags_post_id_tag_id_3b9d1d8d_uniq"
ON "til_app_post_tags" ("post_id", "tag_id");

--
```

```
-- Create indexes to speed up queries on post_id and tag_id
--
CREATE INDEX "til_app_post_tags_post_id_96b51563"
ON "til_app_post_tags" ("post_id");

CREATE INDEX "til_app_post_tags_tag_id_d18818ba"
ON "til_app_post_tags" ("tag_id");

COMMIT;
```

The above has been generated for SQLite. We will discuss how to change the database engine later in this lab.

**TIP**

If you look closely at the SQL output above, you'll notice that Django automatically creates a junction table `til_app_post_tags` to manage the `ManyToManyField` relationship.

**INFO**

Create a circular dependency to see how Django ORM translates to SQL.

3.3 Apply Migrations

To apply our migration to our database, we can use the `migrate` command:

```
python3 manage.py migrate <application|optional>
```

This command will run any migrations from all installed applications that have not yet been applied to our database.

```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, til_app
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
```

```
Applying admin.0002_logentry_remove_auto_add... OK
Applying admin.0003_logentry_add_action_flag_choices... OK
Applying contenttypes.0002_remove_content_type_name... OK
Applying auth.0002_alter_permission_name_max_length... OK
Applying auth.0003_alter_user_email_max_length... OK
Applying auth.0004_alter_user_username_opts... OK
Applying auth.0005_alter_user_last_login_null... OK
Applying auth.0006_require_contenttypes_0002... OK
Applying auth.0007_alter_validators_add_error_messages... OK
Applying auth.0008_alter_user_username_max_length... OK
Applying auth.0009_alter_user_last_name_max_length... OK
Applying auth.0010_alter_group_name_max_length... OK
Applying auth.0011_update_proxy_permissions... OK
Applying auth.0012_alter_user_first_name_max_length... OK
Applying sessions.0001_initial... OK
Applying til_app.0001_initial... OK
```

3.4 Workflow

Applying our models and getting them into our database involves the following setups:

1. Modify the model Python files (`models.py`)
2. Run `python manage.py makemigrations`
3. Run `python manage.py migrate`

STAY IN SYNC

Modifying model files without applying the migrations or modifying the database schema without modifying the models can cause many issues. It is critical to keep these in sync.

TIP

Examine the generated migration code before you run it, especially when complex changes are involved.

Always back up your data before running a migration.

Chapter 4. Choose the Database Engine

By default, Django uses **SQLite**, a standalone file based database as its database engine.

You may also use **MySQL** or **PostgreSQL** for the eventual deployment into AWS.

The database engine used by Django application can be configured in the settings file `settings.py` under the `DATABASES` variable.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.sqlite3',  
        'NAME': BASE_DIR / 'db.sqlite3',  
    }  
}
```

See Django's [database reference](#) for how to change your database engine.

! INFO

SQLite is typically sufficient for your group assignment if you are not familiar with database engine mechanisms.

Chapter 5. Django Admin

Django comes with a powerful admin interface automatically out of the box -

<http://127.0.0.1:8000/admin>. This interface gives us the ability to add, edit and delete models with very little application code.

We will need to be running our application with `python manage.py runserver` to be able to access admin interface.

🔥 DON'T USE DJANGO ADMIN FOR END USERS

The Django admin interface is designed for **site administrators/developers**, not end users. It is a place for your site administrators (like ICT team) to add/edit/delete data and perform site management tasks. Although it is possible to stretch it into something that your end users (staff/manager/website admin) could use, you really should not. It is just

not designed for use by every site visitor.

5.1 Create an Admin User

The admin interface is guarded by `username:password` credentials. The auth application exposes a command to create superusers:

```
python manage.py createsuperuser
```

These users will have access to the admin interface and give an easy way to bootstrap new admin users without needing to drop down to the database directly.

```
$ python manage.py createsuperuser
Username (leave blank to use 'user'): user
Email address: user@user.com
Password:
Password (again):
Superuser created successfully.
```



TIP

You can run `createsuperuser` multiple times to add more superusers.



WARNING

The Django auth application has some default rules for preventing common and insecure passwords. While it is possible to bypass these password restrictions when creating your user, it is not recommended.

Once we have created our user, we will be able to log into the admin interface, via <http://127.0.0.1:8000/admin>, or whatever port you are running your server on.

5.2 Add Models to Django Admin

Our models will not automatically get exposed through the admin interface. We need to

register our models from the TIL application with the admin application before they can be used there.

This can be done through modifying the `til_app/admin.py` file and calling `admin.site.register(<model>)`:

```
from django.contrib import admin

from .models import Post, Tag

admin.site.register(Post)
admin.site.register(Tag)
```

Reload the admin interface, and now you will see the TIL application and exposed models.



TIP

We recommend that you play around with the admin interface and get familiar with the functionalities. You should be able to create, edit and delete data.

5.3 Secure the Django Admin

Since the Django admin gives your site admin special powers that ordinary users don't have, it's good practice to make it extra secure.

Change the Default Admin URL

Django project co-leader Jacob Kaplan-Moss says (paraphrased) that it's an easy additional layer of security to come up with a different name (or even different domain) for the admin.

It also prevents attackers from easily profiling your site. For example, attackers can tell which version of Django you're using, sometimes down to the point-release level, by examining the HTML content of the login screen for `admin/`.

Limit Admin Access Based on IP

Configure your web server to only allow access to the Django admin to certain IP addresses. Look up the instructions for your particular web server.

- Nginx instructions: <https://tech.marksblogg.com/django-admin-logins.html>

An acceptable alternative is to put this logic into middleware. No need to modify the web server configuration (Easy to implement). But introduce slight performance overhead due to the additional middleware layer, not as efficient as doing it at the web server level.

! INFO

You are not required to secure the Django admin for your assignments.

Further Readings

Greenfeld, D., & Roy Greenfeld, A. (2020). Two Scoops of Django 3.x: Best Practices for the Django Web Framework.

The book, while focused on Django 3, offers practical design ideas and best practices for structuring and optimising Django projects. Read following chapters if you are not familiar with Python or Django project:

- Chapter 1: Coding Style
- Chapter 2: The optimal Django Environment Setup
- Chapter 3: How to Lay Out Django Projects
- Chapter 5: Settings and Requirements Files

Credit

This lab content was authored and maintained by **Jiawen Wen**, with materials adapted from prior offerings and updated to align with 2026 delivery. All standards used in this lab (IEEE, ISO/IEC) are referenced for educational purposes under fair use and are available to enrolled students via the university's licensed repository.

